Ficha de Personagem

Aula 26 · Exercício 01 · Como o projeto funciona e o que ele precisa

1. A ideia do projeto

O objetivo é montar uma pequena página onde o usuário cria fichas de personagem de RPG. Cada ficha tem um **nome**, uma **classe** (Guerreiro, Mago ou Ladino) e quatro **atributos**: Força, Agilidade, Inteligência e Resistência. O usuário distribui pontos entre os atributos, pode receber um bônus de acordo com a classe escolhida e, no final, salva a ficha, que aparece numa lista logo abaixo.

Mais do que fazer a página funcionar, o exercício serve para praticar três conceitos importantes de JavaScript: **closures**, **callbacks** e a diferença entre **referência e cópia** de objetos.

2. Como a página deve se comportar

- Cada atributo começa com valor **5** e o jogador tem **10 pontos** livres para distribuir.
- O botão **+** aumenta o atributo em 1 e gasta 1 ponto. Se não houver pontos, nada acontece.
- O botão **-** diminui o atributo em 1 e devolve 1 ponto. Nenhum atributo pode ficar abaixo de 1.
- O contador de **Pontos** na tela sempre mostra quantos pontos ainda restam.
- **Aplicar Bônus** soma os pontos extras da classe escolhida. Só pode ser usado uma vez por ficha.
- **Salvar Ficha** guarda a ficha atual na lista e deixa o formulário pronto para uma ficha nova.
- Se o nome ficar em branco, a ficha recebe o nome padrão **"Herói"**.

Bônus de cada classe:

Classe	Bônus
Guerreiro	+3 Força, +2 Resistência
Mago	+4 Inteligência, +1 Agilidade
Ladino	+3 Agilidade, +2 Inteligência

3. Os três arquivos

index.html tem a estrutura da página: o campo de nome, a seleção de classe, as linhas de atributos com seus botões, os botões de ação e uma área vazia onde as fichas salvas vão aparecer. Todo elemento que o JavaScript precisa ler ou alterar ganha um **id**. Vale escolher ids com um padrão (por exemplo, o nome do atributo dentro do id do botão), porque assim o JavaScript consegue montar o id dentro de um laço em vez de repetir o mesmo código quatro vezes.

style.css cuida só da aparência e não interfere na lógica.

script.js tem toda a lógica. Ele precisa estar **ligado ao HTML** e só pode rodar **depois que a página foi carregada**. Se o script rodar antes, os elementos ainda não existem e a busca pelos ids não encontra nada.

> **Conteúdos:** estrutura HTML, atributo *id*, ligação do script na página e o momento em que ele é executado (atributo *defer* ou script no fim do body).

4. Cada funcionalidade e o que ela precisa

4.1 O rascunho da ficha

Para criar as fichas, usamos um **objeto de rascunho** que representa a ficha que está sendo montada. Ele guarda o nome, a classe e um **objeto interno** com os quatro atributos. Os botões nunca mexem direto na tela: eles alteram o rascunho, e outra função é responsável por mostrar o rascunho na página.

Também precisamos de uma **lista (array)** para guardar as fichas já salvas e de uma variável que indique se o bônus já foi aplicado na ficha atual.

> **Conteúdos:** objetos, objetos dentro de objetos, arrays, variáveis com *let*.

4.2 Controle dos pontos (closure)

Os pontos restantes não devem ficar numa variável solta que qualquer parte do código possa alterar. Para isso, criamos uma **função fábrica** que recebe o total inicial, guarda esse valor numa variável interna e devolve um objeto com três funções: uma para **gastar** pontos (que avisa se deu certo ou não), uma para **devolver** pontos e uma para **consultar** quantos restam.

Como essas três funções foram criadas dentro da fábrica, elas continuam com acesso à variável interna mesmo depois que a fábrica terminou de rodar. Isso é uma **closure**. Quem está de fora só consegue mudar os pontos usando essas funções, então as regras (por exemplo, não gastar mais do que existe) ficam sempre garantidas.

> **Conteúdos:** funções que retornam objetos, escopo de variáveis, closure, retorno de valores booleanos.

4.3 Atualizar a tela

Precisamos de uma função que leia os valores do rascunho e os pontos restantes e escreva cada um no elemento certo da página. Ela é chamada **toda vez que algo muda** e também uma vez quando a página abre, para mostrar os valores iniciais. Ter essa função separada evita espalhar código de tela por todo o arquivo.

> **Conteúdos:** manipulação do DOM, buscar elementos pelo id, alterar o texto de um elemento.

4.4 Botões de + e - (closure de novo)

São oito botões (dois por atributo), mas todos fazem quase a mesma coisa. Em vez de escrever oito funções, criamos **uma função que gera a função de clique**. Ela recebe qual atributo alterar e se deve somar ou subtrair, e devolve a função que vai rodar no clique. Essa função devolvida "lembra" esses dois valores: outra closure.

Dentro dela ficam as regras: para somar é preciso conseguir gastar um ponto; para subtrair, o atributo precisa estar acima de 1, e o ponto é devolvido. Depois de alterar o rascunho, a tela é atualizada.

Para ligar os botões, usamos uma **lista com os nomes dos atributos** e um **laço** que, para cada nome, encontra os dois botões pelo id e registra o evento de clique.

> **Conteúdos:** funções que retornam funções, condicionais, laços de repetição, eventos de clique (*addEventListener*), concatenação de strings para montar ids.

4.5 Bônus de classe (callbacks)

Cada classe tem a sua própria **função de bônus**, que recebe uma ficha e soma os pontos extras nos atributos certos. Depois criamos uma função genérica de aplicar bônus, que recebe a ficha e **uma função como parâmetro**. Ela não sabe qual bônus está aplicando, apenas executa a função que recebeu. Uma função passada como argumento para outra é um **callback**.

Para escolher qual callback usar, criamos um **objeto que associa o nome de cada classe à sua função de bônus**. Assim, o valor escolhido no seletor de classe serve direto como chave para pegar a função certa, sem precisar de vários *if*.

No clique do botão, lemos o nome e a classe da tela, aplicamos o bônus e atualizamos a tela. Para que o bônus não seja usado mais de uma vez, a variável de controle é marcada e o botão e o seletor de classe são desativados até a ficha ser salva.

> **Conteúdos:** callbacks, funções como valores, objetos usados como "mapa" (chave para valor), acesso a propriedades com colchetes, ler valores de input e select, desativar elementos.

4.6 Salvar a ficha (referência x cópia)

Aqui está o ponto mais importante do exercício. Em JavaScript, uma variável que guarda um objeto guarda na verdade uma **referência** a ele. Se colocarmos o próprio rascunho na lista de fichas salvas, a lista e o rascunho vão apontar para o **mesmo objeto**, e qualquer mudança no rascunho também muda a ficha "salva".

Por isso, ao salvar, criamos um **objeto novo** com o nome e a classe e um **novo objeto de atributos**, copiando cada atributo um por um com um laço. Só essa cópia independente vai para a lista.

Depois, a área de fichas da página é limpa e redesenhada: para cada ficha da lista, criamos os elementos do cartão (um contêiner, um título com nome e classe e um parágrafo com os atributos), colocamos um dentro do outro e adicionamos na página.

> **Conteúdos:** referência x valor, cópia de objetos, laço *for...in* para percorrer propriedades, criar elementos com *createElement*, adicionar com *appendChild*, template strings.

4.7 Começar uma nova ficha

Depois de salvar, o formulário precisa voltar ao estado inicial. Para isso, uma função cria um **rascunho novo** com os valores padrão, gera um **novo controle de pontos** chamando de novo a fábrica de closure (que começa com 10 pontos), libera o bônus, limpa o campo de nome, volta a classe para a primeira opção, reativa o botão e o seletor e atualiza a tela.

Repare que, como a ficha salva é uma cópia, trocar o rascunho por um objeto novo não afeta nada do que já foi salvo.

> **Conteúdos:** reatribuir variáveis, reaproveitar a fábrica de closure, reativar elementos da página.

5. Resumo dos conteúdos

Conteúdo	Onde é usado
Objetos e arrays	Guardar o rascunho, os atributos e a lista de fichas salvas.
Closure	Proteger os pontos restantes e gerar as funções dos botões + e -.
Callback	Aplicar o bônus de cada classe com uma única função genérica.
Referência x cópia	Salvar fichas que não mudam quando o rascunho muda.
DOM	Ler campos, mostrar valores e criar os cartões das fichas.
Eventos	Reagir aos cliques em todos os botões.
Laços	Ligar os botões dos atributos, copiar atributos e desenhar a lista.

6. Cuidados comuns

- Esquecer de ligar o script ao HTML, ou rodá-lo antes de a página carregar: nenhum botão funciona.
- Criar variáveis sem *let*: elas viram globais sem querer e podem causar erros difíceis de achar.
- Salvar o próprio rascunho em vez de uma cópia: todas as fichas salvas mudam juntas.
- Não limitar o bônus: o usuário consegue clicar várias vezes e aumentar os atributos sem limite.
- Ler a classe só no botão de bônus: quem salva sem aplicar o bônus fica sempre com a classe padrão.
- Esquecer de atualizar a tela depois de mudar o rascunho: os valores mudam, mas o usuário não vê.